

07/09/26

COL876

Church's
1910's/20's → simply Typed Lambda
Calculus

$\lambda x, \lambda x.t, (t\ v)$
can represent any computable
functions

Alan Turing's Machine : Turing = Lambda Calculus (Church-Turing Hypothesis)

Any systems
that capture
entire mathematics

Girard : natural deduction
Gentzen (+ Intuitionistic Logic)

1934' : Haskell Curry (The main idea)

$p \supset q$ (p implies q)
a formulae in intuitionistic logic

$p \rightarrow q$ (The world of type system)
A function type
(takes in x of type p
returns y of type q)

$\{ \text{provable proposition} \iff \text{computable function} \}$
i.e. propositions as types

some sort of isomorphism?

1960's : Howard → Extends the above idea to proofs as programs (and also $\wedge, \vee$ connectives)

Proposition — Function Type — Program

Π_1 Π_2

$\otimes$: $M : A$ $N : B$

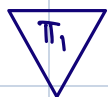
some detour { $(M, N) : A \times B$ $M : A$

$\times I$ (Cross Introduction) $\rightsquigarrow$ Π_1 $M : A$

$\times E$ (Cross Elimination) $\downarrow$ normalised term

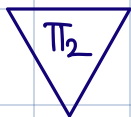
Proof normalisation as evaluation of programs (normalised proof trees)

⊗



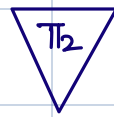
$$x : A \vdash N : B$$

$$\vdash \lambda x. N : A \rightarrow B$$



$$M : A$$

cut
rule



$$M : A \vdash N[x/M] : B$$

Function
that takes
a pair and
hips it

$$\vdash (\lambda x. N) M : B$$

($\rightarrow E$)

(some $\rightarrow$
elimination)

(we cut and paste
the π_2 proof
everywhere replace
 x 's occurrences in
 N with M)

⊗

$$z : B \times A$$

$$z : B \times A$$

$$\pi_2 z : A$$

$$\pi_1 z : B$$

$$(\pi_2 z, \pi_1 z) : A \times B$$

$$\lambda z. (\pi_2 z, \pi_1 z) : (B \times A) \rightarrow (A \times B)$$

$$\lambda z. (\pi_2 z, \pi_1 z) (y, x) : A \times B$$

Actually
providing the
two π 's

$$y : B$$

$$x : A$$

$$(y, x) : B \times A$$

But using cut rule we can just replace
every occuren of z with (y, x) to get:
(which is like saying f applied to (x, y) is $f(x, y)$)

$$y : B \quad x : A$$

$$y : B \quad x : A$$

$$(y, x) : B \times A$$

$$(y, x) : B \times A$$

$$\pi_2 (y, x) : A$$

$$\pi_1 (y, x) : B$$

$$(\pi_2 (y, x), \pi_1 (y, x)) : A \times B$$

suppose we
are applying this
to $x : A, y : B$

$$g ((y, x) : B \times A) = (x, y) : A \times B$$

There is no rule to
infer types unless (x, y)
is provided explicitly

$$x : A$$

$$y : B$$

$$(x, y) : A \times B$$

universe/

Lean : dependent Intuitionistic logic

↓

• To avoid Russell's Paradox with types

$\text{Nat} : \text{Type}$
 $\text{Type} : \text{Type } 1$
 $\text{Type } 1 : \text{Type } 2$
 $\vdots$

universes on something

family of simple types

• polymorphism in types : $\text{List} \cdot \{u\} \{ \alpha : \text{Type } u \} \text{Type } u$

Not only is this polymorphic in universe type u

(which has basic types)

(List_{Nat} , $\text{List}_{\text{BinTree}}$, List_{Str})

It can also be polymorphic across other universe types

($\text{List Type } 1$)

universe $u \ v$

def createprod $\{ \alpha : \text{Type } u \} \{ \beta : \alpha \rightarrow \text{Type } v \}$

$\downarrow$ $(a : \alpha) (b : \beta a) : \alpha \times \beta a := (a, b)$

Type hierarchy well-founded at Type 0

Lean can work in higher order Type universes